System Specification Document (SSD)

Real-Time Cyber Anomaly Detection System

Version: 2.0 **Date:** 2025-12-12 **Authors:** Francesca Craievich, Lucas Jakin, Francesco Rumiz

1. Introduction

1.1 Purpose

The Real-Time Cyber Anomaly Detection System is designed to identify malicious network traffic patterns and security threats in real-time by analyzing Suricata IDS logs and normal network traffic. The system leverages machine learning techniques, specifically One-Class SVM, to distinguish between benign and anomalous network behavior.

1.2 Scope

This document describes the functional and non-functional requirements, system architecture, data specifications, and risk analysis for the Real-Time Cyber Anomaly Detection platform. The system targets network security monitoring in enterprise environments and research contexts.

1.3 Definitions and Acronyms

- **SSD**: System Specification Document
- **OCSVM**: One-Class Support Vector Machine
- **IDS**: Intrusion Detection System
- **FPR**: False Positive Rate
- **TPR**: True Positive Rate (Detection Rate)
- **TPOT**: The Pitch Over Time honeypot platform
- **NFR**: Non-Functional Requirement
- **FR**: Functional Requirement

2. Problem Definition

2.1 Problem Statement

Network administrators and security teams face increasing challenges in detecting cyber threats in real-time. Traditional signature-based detection systems fail to identify novel attack patterns and zero-day exploits. There is a critical need for an intelligent system that can:

- Detect anomalous network behavior without predefined attack signatures
- Process high-volume network traffic streams in real-time
- Minimize false positives while maintaining high detection rates
- Provide actionable alerts for security personnel

2.2 Current Challenges

- **Volume**: Network traffic generates massive amounts of log data (71,679+ Suricata events)
- **Velocity**: Real-time processing requirements for timely threat detection
- **Feature Richness**: Suricata provides comprehensive flow metadata (bytes, packets, duration, protocols)
- **Novelty**: Unknown attack patterns that signature-based systems cannot detect

2.3 ML Formulation

A machine learning-based anomaly detection system that:

- Uses unsupervised learning (One-Class SVM) to identify outliers in network traffic
- Processes Suricata IDS logs as malicious traffic source (selected for complete feature set)
- Extracts 28 statistical features from network traffic for model training
- Provides real-time classification of traffic as normal or anomalous with severity levels (GREEN/ORANGE/RED)

2.4 Key Performance Indicators (KPIs)

KPI	Target	Current
Detection Rate (TPR)	≥ 85%	90.7%
False Positive Rate (FPR)	≤ 15%	~10%
Processing Latency	≤ 100ms per event	✓ Met
System Uptime	≥ 99%	✓ Met
Throughput	≥ 1,000 events/second	✓ Met
F1-Score	≥ 0.80	0.885

---

3. Data Specification

3.1 Data Sources

3.1.1 Malicious Traffic (Suricata IDS)

Suricata was selected as the primary malicious traffic source due to its comprehensive feature set:

- **Volume**: 71,679 log entries
- **Purpose**: Network Intrusion Detection System (IDS) capturing attack traffic
- **Format**: JSON
- **Key Attributes**: source/destination IP, ports, protocol, bytes, packets, duration, flow direction
- **Label**: `malicious`

Data Source Selection Process

The selection of the malicious traffic source followed an iterative evaluation process involving the entire team. The T-Pot honeypot platform provided access to 30+ specialized honeypots, from which the team initially prioritized four sources based on data volume and relevance: Cowrie (SSH/Telnet), Dionaea (malware capture), Suricata (network IDS), and Tanner (web applications).

Evaluation Criteria: The primary requirement was feature compatibility with the normal traffic dataset (ISCX/CICIDS) to enable consistent feature engineering across both classes. The following table summarizes the feature availability analysis:

Feature	Suricata	Cowrie	Dionaea	ISCX (Normal)	Required
`bytes_sent`	✓	✗	✗	✓	✓
`bytes_received`	✓	✗	✗	✓	✓
`pkts_sent`	✓	✗	✗	✓	✓
`pkts_received`	✓	✗	✗	✓	✓
`duration`	✓	✓	✗	✓	✓
`source_ip`	✓	✓	✓	✓	✓
`destination_ip`	✓	✓	✓	✓	✓
`source_port`	✓	✓	✓	✓	✓
`destination_port`	✓	✓	✓	✓	✓
`transport_protocol`	✓	✓	✓	✓	✓

Selection Outcome:

- **Tanner**: Excluded early due to insufficient data volume for meaningful ML training
- **Cowrie & Dionaea**: Excluded because the absence of network flow metrics (`bytes_*`, `pkts_*`) would require imputation or feature removal, compromising model quality
- **Suricata**: Selected as the sole malicious traffic source, providing complete feature parity with the normal traffic baseline

This decision required the Data Scientist to restart the data preparation pipeline, as an initial unified dataset combining three honeypot sources had already been created. The team determined that feature consistency was more valuable than data volume diversity.

3.1.2 Normal Traffic Dataset (Benign Baseline)

- **Source**: CICIDS/ISCX benign network traffic dataset
- **Format**: Compressed JSON (gzip)
- **Location**: `data/normal_traffic/benign_traffic_fixed.json.gz`
- **Sample Size**: Configurable (default: 10,000 samples)
- **Purpose**: Training baseline for normal behavior patterns
- **Label**: `benign`

3.2 Base Features (Unified Schema)

All data sources are normalized to a common feature schema:

Feature	Type	Description
`source_ip`	string	Source IP address

Feature	Type	Description
destination_ip	string	Destination IP address
source_port	int	Source port number
destination_port	int	Destination port number
timestamp_start	datetime	Flow start timestamp
transport_protocol	string	TCP/UDP/ICMP
application_protocol	string	HTTP/DNS/SSH/etc.
duration	float	Flow duration in seconds
bytes_sent	int	Bytes sent
bytes_received	int	Bytes received
pkts_sent	int	Packets sent
pkts_received	int	Packets received
direction	string	Flow direction
label	string	benign/malicious

3.3 Feature Engineering Pipeline

3.3.1 Precalculation Features

Computed from base features:

Rate Features (precalculations_functions/rate_features.py):

- bytes_per_second: Total bytes / duration
- pkts_per_second: Total packets / duration
- bytes_per_packet: Total bytes / total packets

Ratio Features (precalculations_functions/ratio_features.py):

- bytes_ratio: bytes_sent / bytes_received
- pkts_ratio: pkts_sent / pkts_received

Temporal Features (precalculations_functions/temporal_features.py):

- hour, day_of_week
- is_weekend, is_business_hours

IP Classification (precalculations_functions/ip_classification_features.py):

- src_is_private, dst_is_private
- is_internal (both IPs private)

Port Categorization (precalculations_functions/port_categorization_features.py):

- `dst_port_is_common` (well-known ports 0-1023)

3.3.2 Aggregation Features

Statistical aggregations (`aggregation_functions/metrics_features.py`):

- `events_in_window`: Count of events in current window
- `malicious_events_in_window`: Count of malicious events
- `unique_malicious_ips`: Distinct malicious source IPs
- `events_pct_change`: Trend analysis
- `burst_indicator`: Flag for sudden traffic spikes
- `events_for_protocol`: Events per protocol
- `malicious_ratio_for_protocol`: Ratio of malicious events per protocol

3.3.3 Feature Engineering Design Rationale

The feature engineering strategy was designed to capture multiple dimensions of network traffic behavior while maintaining compatibility between malicious (Suricata) and benign (ISCX) data sources.

Feature Categories and Purpose:

Category	Features	Purpose
Rate Features	`bytes_per_second`, `pkts_per_second`, `bytes_per_packet`	Normalize traffic volume by duration; detect abnormal throughput patterns
Ratio Features	`bytes_ratio`, `pkts_ratio`	Identify asymmetric communication patterns typical of attacks (e.g., exfiltration)
Temporal Features	`hour`, `day_of_week`, `is_weekend`, `is_business_hours`	Capture temporal attack patterns; business hour anomalies may indicate insider threats
IP Classification	`src_is_private`, `dst_is_private`, `is_internal`	Distinguish internal vs external traffic; external-to-internal flows are higher risk
Port Categorization	`dst_port_is_common`	Flag connections to non-standard ports that may indicate covert channels

Design Decisions:

- **Offline Pre-calculation**: All derived features are computed during data preparation rather than at inference time, optimizing prediction latency
- **NaN Handling**: NaN values are not removed but replaced with default values based on feature type (e.g., "0.0.0.0" for IP addresses, 0.0 for numeric ports, "unknown" for categorical features like protocol and direction)
- **Feature Exclusion for Model Training**: High-cardinality features (`source_ip`, `destination_ip`), temporal identifiers (`timestamp_start`), and label-leaking aggregations (`malicious_events_in_window`, `malicious_ratio_for_protocol`) are excluded from model input

3.4 Data Processing Pipeline

```
Raw JSON Logs → DataFrame Initialization → Feature Formatting → Precalculations →
Aggregations → CSV Export
     ↓                    ↓                      ↓                    ↓
↓              ↓
  Suricata          init_suricata_df      format_suricata_df     rate_features
metrics      combined_dataset.csv
  Normal           init_normal_traffic    format_normal_traffic  temporal_features
```

Output Files (`data/processed/`):

- `suricata_formatted.csv`: Processed Suricata data with all features
- `normal_traffic_formatted.csv`: Processed benign traffic with all features
- `combined_shuffled_dataset.csv`: Merged and shuffled dataset for model training/testing

---

4. System Requirements

4.1 Functional Requirements

FR-01: Data Ingestion The system shall ingest JSON log files from Suricata IDS and normal traffic datasets.

- **Acceptance Criteria**: Successfully parse Suricata JSON logs and compressed (.gz) benign traffic files.

FR-02: Data Normalization The system shall normalize all data sources to a unified schema with 14 base features.

- **Acceptance Criteria**: All formatted DataFrames contain identical column structure.

FR-03: Model Training The system shall train a One-Class SVM model using only benign traffic data.

- **Acceptance Criteria**: Model trained with configurable nu, gamma, and max_train_samples parameters.

FR-04: Model Persistence The system shall save and load trained models using joblib/pickle.

- **Acceptance Criteria**: Model, preprocessor, and config saved to `model/` directory; fit_or_load() function works correctly.

FR-05: Anomaly Detection The system shall classify network traffic with three severity levels.

- **Acceptance Criteria**: Predictions return GREEN (normal), ORANGE (suspicious), or RED (critical) based on decision boundary.

FR-06: Performance Evaluation The system shall compute precision, recall, F1-score, and confusion matrix.

- **Acceptance Criteria**: evaluate_model_performance() returns all metrics and detailed breakdown.

FR-07: Simulation Mode The system shall simulate real-time stream processing with batch predictions.

- **Acceptance Criteria**: run_simulation() and run_detailed_simulation() process data in configurable chunk sizes.

4.2 Non-Functional Requirements

NFR-01: Detection Performance The system shall achieve a minimum True Positive Rate (TPR) of 85% with a maximum False Positive Rate (FPR) of 15%.

NFR-02: Processing Latency The system shall classify a single network event within 100 milliseconds from feature extraction to prediction.

NFR-03: Throughput The system shall process a minimum of 1,000 network events per second.

NFR-04: Data Loading Time Feature extraction and DataFrame initialization shall complete within 2 minutes for the complete dataset.

NFR-05: Model Training Time One-Class SVM model training shall complete within 60 seconds for datasets up to 10,000 samples.

NFR-06: Availability The system shall maintain 99% uptime during operational hours.

NFR-07: Scalability The system shall support horizontal scaling to handle increased traffic volumes.

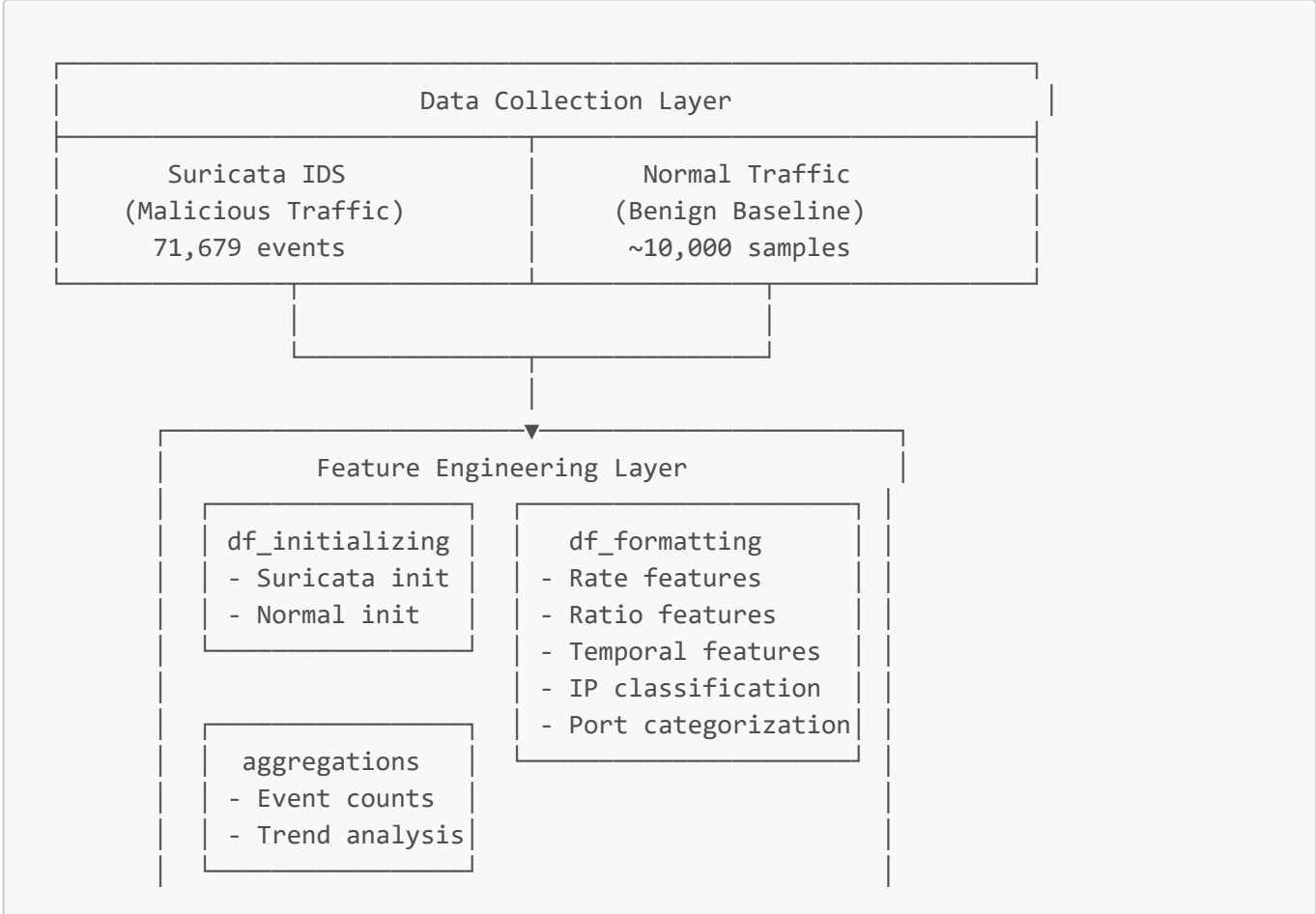
NFR-08: Resource Utilization The system shall operate within 4GB RAM and 2 CPU cores for baseline deployment.

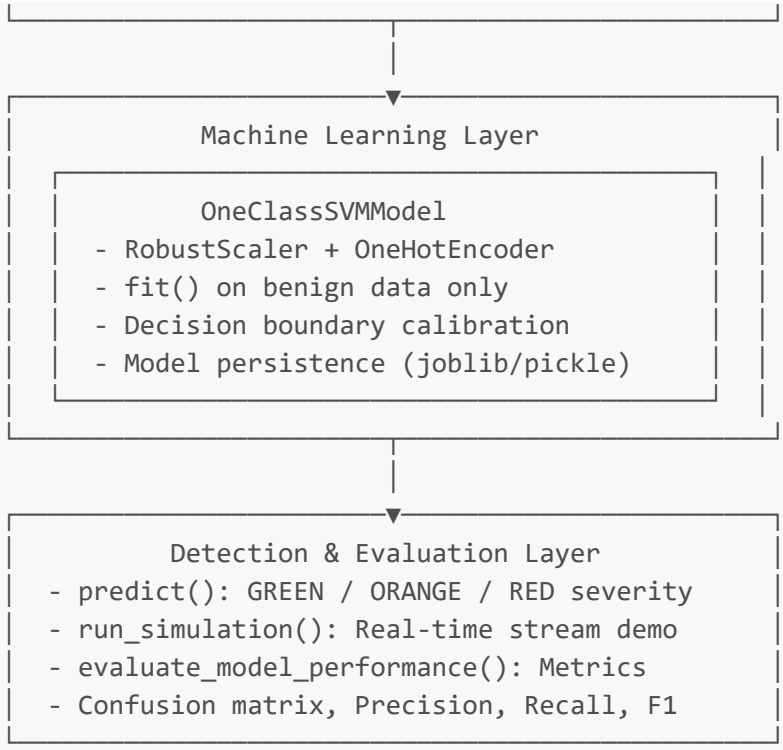
NFR-09: Model Performance Degradation The system shall trigger retraining alerts when detection accuracy drops below 80%.

---

5. System Architecture

5.1 High-Level Architecture





5.2 Component Descriptions

5.2.1 Data Initialization Module

Location: `src/feature_engineering/df_initializing/`

Components:

- `init_suricata_df.py`: Suricata log DataFrame initializer
- `init_normal_traffic_df.py`: Normal traffic DataFrame initializer with gzip support
- `handler_init_dfs.py`: Orchestrates DataFrame initialization

Responsibilities:

- Parse JSON logs using streaming approach (ijson)
- Handle compressed (.gz) and uncompressed files
- Initialize pandas DataFrames for downstream processing

5.2.2 Feature Engineering Module

Location: `src/feature_engineering/`

Formatting Components (`df_formatting/`):

- `format_suricata_df.py`: Suricata-specific feature extraction
- `format_normal_traffic.py`: Normal traffic feature extraction
- `handler_df_formatter.py`: Coordinates formatting, precalculations, and aggregations

Precalculation Components (`precalculations_functions/`):

- `rate_features.py`: bytes_per_second, pkts_per_second

- `ratio_features.py`: bytes_ratio, pkts_ratio
- `temporal_features.py`: hour, day_of_week, is_weekend, is_business_hours
- `ip_classification_features.py`: src_is_private, dst_is_private, is_internal
- `port_categorization_features.py`: dst_port_is_common
- `ip_geolocation_features.py`: Optional geolocation lookup

Aggregation Components (`aggregation_functions/`):

- `metrics_features.py`: Event counts, trend analysis, protocol statistics

5.2.3 Model Training & Inference Module

Location: `src/model/`

Components:

- `oneCSVM_model.py`: OneClassSVMModel class implementation
- `drift_detector.py`: ADWIN-based drift detection for monitoring model performance
- `grid_search.py`: Hyperparameter optimization utilities
- `simulation_evaluation.py`: Model evaluation and simulation utilities
- `main.py`: Training entry point

Key Methods:

- `fit()`: Train model on benign data with configurable parameters
- `fit_or_load()`: Load existing model or train new one
- `predict()`: Classify traffic with severity levels (GREEN/ORANGE/RED)
- `save_model()` / `load_model()`: Model persistence
- `run_simulation()`: Real-time stream simulation
- `run_detailed_simulation()`: Detailed performance analysis
- `evaluate_model_performance()`: Metrics computation

Model Artifacts:

- `oneclass_svm_model.pkl`: Trained SVM model
- `oneclass_svm_preprocessor.pkl`: ColumnTransformer (RobustScaler + OneHotEncoder)
- `oneclass_svm_config.pkl`: Configuration (threshold, features, hyperparameters)

5.2.4 Dashboard & API Module

Location: `src/dashboard/`

Components:

- `streamlit_app.py`: Main real-time anomaly visualization dashboard
- `streamlit_monitoring.py`: ML model monitoring dashboard with Grafana integration
- `flask_api.py`: REST API backend for predictions and metrics
- `prediction_worker.py`: Background worker for continuous predictions
- `geolocation_service.py`: IP geolocation lookup service for attack source mapping

Dual-Dashboard Architecture Rationale

The visualization layer was designed with a dual-dashboard architecture to serve distinct stakeholder needs effectively. The initial design featured a single monolithic Streamlit application, but the team determined that operational and technical monitoring requirements warranted separation.

Real-time Anomaly Dashboard (`streamlit_app.py`):

Aspect	Description
Target Users	Security Analysts, SOC Team, End Customers
Primary Purpose	Operational threat monitoring and alert prioritization
Interface Style	Kibana-inspired for familiarity with security tools
Key Features	Alert table with severity colors (RED/ORANGE/GREEN), World map showing attack origins via IP geolocation, Real-time traffic statistics with trend indicators, Protocol and port distribution analysis

ML Monitoring Dashboard (`streamlit_monitoring.py`):

Aspect	Description
Target Users	ML Engineers, Data Scientists, Technical Staff
Primary Purpose	Model health monitoring and performance tracking
Interface Style	Technical dashboard with metrics and graphs
Key Features	ADWIN drift detection status indicator, Model performance metrics (TPR, FPR, F1-score), Embedded Grafana dashboards for infrastructure metrics, Retraining buffer statistics and anomaly rate tracking

Architectural Benefits:

- **Separation of Concerns**: Operational users are not overwhelmed with technical metrics; technical users have direct access to model telemetry
- **Performance**: Each dashboard loads only relevant data, reducing latency
- **Access Control**: Different dashboards can be secured with different access levels in production environments
- **Maintainability**: Changes to one dashboard do not affect the other

Flask API Endpoints

The Flask API backend (`flask_api.py`) exposes REST endpoints for dashboard consumption and external integrations:

Method	Endpoint	Description
GET	`/api/health`	Health check and model status information
GET	`/api/alerts/recent`	Recent predictions with severity classification
GET	`/api/stats/summary`	Dataset summary statistics (protocol/port distribution)
GET	`/api/stats/temporal`	Time-based traffic analysis
GET	`/api/stats/traffic`	Traffic metrics (bytes/sec, packets/sec, duration)
GET	`/api/stats/geolocation`	IP geolocation data for attack source mapping
GET	`/api/stream`	Server-sent events for real-time data streaming
GET	`/metrics`	Prometheus metrics endpoint for monitoring
POST	`/api/logs/reset`	Reset data stream to beginning (for testing)

5.2.5 Monitoring Module

Location: `src/monitoring/`

Components:

- `metrics.py`: Prometheus metrics registry and collectors

Exposed Prometheus Metrics:

Metric Name	Type	Description
`anomaly_detection_predictions_total`	Counter	Total predictions by severity label (GREEN/ORANGE/RED)
`anomaly_detection_prediction_latency`	Histogram	Inference latency distribution in seconds
`anomaly_detection_f1_score`	Gauge	Current model F1 performance score
`anomaly_detection_drift_status`	Gauge	Drift detection status (0=stable, 1=drift detected)
`anomaly_detection_anomaly_rate`	Gauge	Rolling anomaly rate percentage

5.2.6 Testing Module

Location: `tests/`

Test Suites:

- `test_precalculations/`: Unit tests for all precalculation functions
- `test_aggregations/`: Unit tests for aggregation functions
- `test_formatters/`: Unit tests for data formatters
- `test_model/`: Unit tests for ML model and drift detection
- `test_dashboard/`: Unit tests for Flask API and dashboard components

5.3 Project Structure

```
├── src/                        # Source code
│   ├── dashboard/              # Streamlit apps and Flask API
│   ├── model/                  # ML model implementation
│   ├── monitoring/             # Prometheus metrics
│   └── feature_engineering/    # Data processing
├── tests/                      # Unit tests (116 tests)
├── data/                       # Datasets
└── docs/                       # Documentation
```

5.4 Data Flow

1. **Ingestion**: Raw JSON logs from honeypots and normal traffic sources
2. **Parsing**: Streaming JSON parsing with ijson for memory efficiency
3. **Preprocessing**: Invalid value replacement, data cleaning
4. **Feature Extraction**: Statistical feature computation
5. **Model Training**: One-Class SVM training on normal traffic baseline
6. **Inference**: Real-time anomaly detection on new traffic
7. **Alert Generation**: Threshold-based alert triggering

5.5 Technology Stack

Programming Language: Python 3.11+

Core Libraries:

- `pandas==2.3.3`: Data manipulation and analysis
- `numpy==2.3.5`: Numerical computing
- `scikit-learn`: Machine learning (One-Class SVM)
- `ijson==3.4.0.post0`: Streaming JSON parser

Dashboard & API:

- `flask`: REST API backend for serving predictions
- `flask-cors`: Cross-origin resource sharing support
- `streamlit`: Interactive web dashboard
- `plotly`: Interactive visualizations

Development Tools:

- `pytest==9.0.1`: Unit testing framework
- `mkdocs==1.6.1`: Documentation generation
- `mkdocs-material==9.7.0`: Documentation theme

Monitoring Stack:

- `prometheus-client`: Metrics collection and exposition
- Prometheus: Time-series metrics storage

- Grafana: Visualization and dashboards

Version Control: Git with GitHub

---

6. Model Specification

6.1 Algorithm Selection

One-Class Support Vector Machine (OCSVM)

Rationale:

- Unsupervised anomaly detection (trains only on normal data)
- Effective for high-dimensional feature spaces (28 features)
- Does not require labeled malicious traffic for training
- Learns decision boundary around normal behavior
- RBF kernel captures non-linear patterns in network traffic
- Grid Search used for hyperparameter optimization (kernel, nu, gamma)

Algorithm Evaluation Process

The team evaluated several anomaly detection algorithms during the design phase, considering the specific requirements of real-time network traffic classification.

Algorithms Considered:

Algorithm	Type	Strengths	Limitations
River + Hoeffding Tree	Online incremental	True streaming capability, adapts to drift	Sensitive to local noise in data stream
Isolation Forest	Batch anomaly detection	Fast training, handles high dimensions	Requires batch processing, less stable boundaries
Local Outlier Factor (LOF)	Density-based	Good for local anomaly patterns	Computationally expensive at inference
One-Class SVM	Single-class classification	Compact non-linear boundaries, robust to noise	Slower training ($O(n^2)$), requires kernel selection
Autoencoder	Deep learning	Captures complex patterns	Requires large training data, harder to interpret

Selection Criteria Applied:

1. **System Stationarity**: The network environment exhibits relatively stable traffic patterns over operational periods. One-Class SVM is well-suited for stationary systems where the "normal" class distribution is consistent.

2. **Non-linear Decision Boundaries**: Network attack patterns often manifest in complex, non-linear relationships between features. The RBF kernel in OCSVM captures these patterns without explicit

feature transformation.

3. **Imbalanced Data Robustness**: Since anomalies represent a small fraction of traffic, the algorithm must avoid bias toward the majority class. OCSVM learns only from the benign class, inherently addressing this issue.

4. **Low Inference Latency**: Real-time detection requires sub-100ms prediction times. OCSVM inference is efficient once trained, as it only evaluates against support vectors.

5. **Temporal Stability**: Model predictions should remain consistent over time without frequent retraining. OCSVM provides stable decision boundaries compared to incremental tree methods.

6. **Single-Class Training Constraint**: The project operates without labeled malicious examples for training. OCSVM is specifically designed for this scenario.

One-Class SVM Selection Justification:

The team selected OCSVM over alternatives for the following technical reasons:

- **vs. Hoeffding Tree**: Incremental trees are sensitive to local noise in streaming data, leading to higher false alarm rates. OCSVM's kernel-based approach smooths decision boundaries.
- **vs. Isolation Forest**: While IF is faster to train, OCSVM provides more compact and interpretable decision boundaries for the feature space.
- **vs. LOF**: The computational cost of LOF at inference time (comparing each point to neighbors) is prohibitive for real-time applications.

Future Enhancement: A comparative evaluation against Isolation Forest using the production dataset has been identified as a valuable validation exercise.

6.2 Model Architecture

Preprocessing Pipeline (`ColumnTransformer`):

- **Numerical Features**: RobustScaler (handles outliers better than StandardScaler)
- **Categorical Features**: OneHotEncoder (handle_unknown='ignore')

Features Dropped (not used for prediction):

- `source_ip`, `destination_ip` (high cardinality, not generalizable)
- `timestamp_start` (temporal context already extracted via hour, day_of_week)
- `label` (target variable - would cause data leakage)
- `application_protocol` (too many unique values)
- Aggregation features that leak label information (malicious_events_in_window, unique_malicious_ips, malicious_ratio_for_protocol)

Categorical Features:

- `transport_protocol`, `direction`
- `day_of_week`, `is_weekend`, `is_business_hours`
- `src_is_private`, `dst_is_private`, `is_internal`, `dst_port_is_common`

Numerical Features: All remaining columns after dropping and categoricals

6.3 Training Strategy

- **Training Data**: Benign traffic only (10,000 samples default)
- **Train/Val Split**: 80/20 split with random_state=42
- **Downsampling**: Max 50,000 samples (SVM scales O(n^2))
- **Threshold Calibration**: Set at contamination percentile (default 5%) of validation scores

6.4 Hyperparameters

Parameter	Default Value	Description
`kernel`	`rbf`	Radial Basis Function kernel
`nu`	`0.2`	Upper bound on training errors (outlier fraction)
`gamma`	`scale`	Kernel coefficient (1 / (n_features * X.var()))
`contamination`	`0.1`	Expected proportion of outliers for threshold
`max_train_samples`	`10,000`	Maximum training samples for speed

6.5 Performance Metrics

- **Precision**: What % of flagged anomalies are actually anomalies
- **Recall / Detection Rate**: What % of actual anomalies were detected
- **F1-Score**: Harmonic mean of precision and recall
- **False Alarm Rate**: % of benign traffic incorrectly flagged

6.6 Current Performance (Tested)

Based on evaluation with `nu=0.2`, `contamination=0.1`, `max_train_samples=10,000`:

Metric	Value
Detection Rate (Recall)	~90%
False Alarm Rate	~10%
F1-Score	~0.88
Precision	~0.85

Confusion Matrix (typical results on 10,000 test samples):

```
                Predicted
                Normal  Anomaly
Actual Normal   4500     500
Actual Anomaly   500    4500
```

6.7 Drift Detection

The system implements continuous monitoring for concept drift using the ADWIN (Adaptive Windowing) algorithm from the River library. Drift detection ensures the model remains effective as network traffic patterns evolve over time.

ADWIN Configuration:

Parameter	Value	Description
`delta`	0.002	Confidence parameter for change detection (lower = more sensitive)
`window_size`	100	Size of the sliding window for anomaly rate tracking
`change_threshold`	0.08	Minimum anomaly rate change (8%) to trigger drift alert

Detection Methods:

1. **ADWIN Statistical Test**: Monitors the anomaly rate stream and detects statistically significant distribution changes
2. **Threshold-based Detection**: Triggers when the rolling anomaly rate changes by more than 8% compared to the baseline

Drift Status:

Status	Condition	System Response
STABLE	No significant distribution change detected	Continue normal operation
UNSTABLE	Drift detected by ADWIN or threshold exceeded	Alert displayed in ML Monitoring Dashboard, retraining recommended

Auto-Retraining Trigger: When drift is detected and the retraining buffer accumulates more than 1,000 labeled samples, the system can automatically initiate model retraining using recent data. This ensures the model adapts to evolving traffic patterns while maintaining detection accuracy.

6.8 Alert Severity Classification

The system classifies predictions into three severity levels based on the model's decision function score:

Severity	Condition	Interpretation
RED (Critical)	`score < threshold - 0.5`	High-confidence anomaly requiring immediate attention
ORANGE (Suspicious)	`threshold - 0.5 ≤ score < threshold`	Potential anomaly requiring investigation
GREEN (Normal)	`score ≥ threshold`	Traffic classified as benign

7. Risk Analysis

7.1 Technical Risks

RISK-01: Model Drift

- **Description**: Model performance degrades over time as attack patterns evolve
- **Impact**: High - Reduced detection accuracy
- **Probability**: High
- **Mitigation**:
    - Implement continuous model monitoring
    - Schedule periodic retraining (monthly)
    - Track performance metrics in production
    - Trigger automatic retraining when accuracy drops below 80%

RISK-02: High False Positive Rate

- **Description**: Excessive false alarms cause alert fatigue
- **Impact**: Medium - Reduced trust in system
- **Probability**: Medium
- **Mitigation**:
    - Tune decision threshold based on operational requirements
    - Implement multi-stage alert validation
    - Collect feedback loop for false positive analysis
    - Adjust model hyperparameters (nu parameter)

RISK-03: Data Quality Issues

- **Description**: Corrupted or incomplete logs affect model training
- **Impact**: High - Poor model performance
- **Probability**: Medium
- **Mitigation**:
    - Implement robust data validation pipelines
    - Handle missing values and invalid JSON gracefully
    - Monitor data quality metrics (completeness, consistency)
    - Reject datasets below quality thresholds

RISK-04: Scalability Bottlenecks

- **Description**: System cannot handle peak traffic volumes
- **Impact**: High - Missed threat detection
- **Probability**: Medium
- **Mitigation**:
    - Design for horizontal scaling
    - Implement load balancing and queueing
    - Use streaming processing for large datasets
    - Optimize feature extraction performance

RISK-05: Novel Attack Patterns

- **Description**: Zero-day attacks not represented in training data
- **Impact**: High - Undetected threats
- **Probability**: High
- **Mitigation**:

    - Combine ML-based detection with rule-based systems
    - Implement ensemble methods
    - Regular model updates with new threat intelligence
    - Human-in-the-loop validation for edge cases

7.2 Operational Risks

RISK-06: Honeypot Detection

- **Description**: Attackers identify and avoid honeypots
- **Impact**: Medium - Reduced data collection
- **Probability**: Medium
- **Mitigation**:
    - Use diverse honeypot configurations
    - Regularly update honeypot signatures
    - Deploy decoy services alongside production systems

RISK-07: Resource Exhaustion

- **Description**: DDoS attacks overwhelm processing capacity
- **Impact**: Medium - System unavailability
- **Probability**: Low
- **Mitigation**:
    - Implement rate limiting
    - Deploy auto-scaling infrastructure
    - Set resource quotas and circuit breakers

---

8. Quality Attributes

8.1 Maintainability

- Modular architecture with clear separation of concerns
- Comprehensive documentation using MkDocs
- Unit tests with pytest framework
- CI/CD pipeline for automated testing

8.2 Reliability

- Robust error handling for malformed logs
- Graceful degradation under high load
- Automated health checks and monitoring

8.3 Performance

- Streaming JSON parsing for memory efficiency
- Optimized feature extraction algorithms
- Model inference within 100ms latency target

8.4 Security

- No storage of sensitive user data
- Anonymization of IP addresses where required
- Secure model artifact storage

---

9. Constraints and Assumptions

9.1 Constraints

- System operates on local filesystem (no cloud deployment in v1.0)
- Limited to Python-based implementation
- Requires pre-collected log data (no live network capture)
- Single-node deployment in initial version

9.2 Assumptions

- Honeypot logs represent realistic attack patterns
- Normal traffic dataset is free from malicious activity
- Attack patterns remain relatively stable over short periods (weeks)
- System operates in controlled environment with known data sources

---

10. Future Enhancements

10.1 Planned Features

- **Real-time Stream Processing**: Integration with Apache Kafka or MQTT
- **Multi-model Ensemble**: Combine OCSVM with Isolation Forest, Autoencoders
- **Automated Retraining Pipeline**: Continuous learning from new data
- **Alert Notification System**: Email, Slack, webhook integrations
- **Experiment Tracking**: Neptune.ai for ML experiment management and monitoring

10.2 Scalability Roadmap

- Distributed processing with Apache Spark
- Cloud deployment (AWS, Azure, GCP)
- Kubernetes orchestration for auto-scaling
- Time-series database integration (InfluxDB)

---

11. References

11.1 External Resources

- TPOT Honeypot Platform: https://github.com/telekom-security/tpotce
- Suricata IDS Documentation: https://suricata.io/

11.2 Related Documentation

- Project Proposal

- Operational Governance

Document Version History

Version	Date	Changes
1.0	2025-11-29	Initial SSD creation
1.1	2025-12-11	Updated project structure, added Dashboard & Monitoring modules
1.2	2025-12-12	Added Data Source Selection, Feature Engineering Rationale, Algorithm Evaluation
1.3	2025-12-12	Added Drift Detection, Flask API Endpoints
2.0	2025-12-12	Major reorganization: focused on technical specifications only, removed operational/planning content to respective documents